

ScholAR

Local LLM Research Paper Assistant

Milestone 1 Report

April 30, 2026

Project goal	Search, process, view, and study arXiv papers with a local Qwen model.
Milestone focus	A simple working MVP with local storage, PDF processing, study goals, and grounded chat.
Status	Core flow implemented and tested with Attention Is All You Need.

1. Project Overview

ScholAR is a local research paper assistant for students, engineers, and researchers. The goal is to make reading papers less stressful and more structured. A user can search for papers from arXiv, select one paper, download and process the PDF, view the paper, and study it with help from a local Ollama Qwen model.

This milestone focuses on a simple working system. The project does not include login, deployment, recommendations, fine-tuning, or a knowledge graph yet.

2. Real Problem

Research papers are difficult to read because they are long, dense, and often assume background knowledge. Many readers understand the abstract but struggle to connect the method, experiments, results, and implementation details.

This affects students, applied AI engineers, independent researchers, and people who need to study new papers quickly. It is especially hard when a person is reading outside their main area of expertise.

3. Why Existing Solutions Fall Short

Normal PDF readers show the paper, but they do not guide learning. General chatbots can summarize text, but they may send private documents to cloud services, lose source grounding, or answer in a way that is not clearly tied to the paper.

Academic search tools are useful for finding papers, but they usually stop at metadata, citations, and abstracts. They do not turn a selected paper into a study session.

4. Proposed Approach

ScholAR combines four simple parts:

- arXiv search to find papers.
- Local PDF download and text extraction.
- Simple chunking and keyword retrieval.
- Local Qwen through Ollama for study goals and paper chat.

The assistant answers questions using retrieved paper chunks and returns page citations. If the local model is unavailable or too slow, the system still returns fallback study goals and grounded extractive answers.

5. Milestone 1 Scope

Milestone 1 includes the core end-to-end workflow:

- Search arXiv papers from the home page.
- Open a paper card and view paper details.
- Prepare the paper by downloading the PDF.
- Extract text page by page with PyMuPDF.
- Save metadata, pages, and chunks locally.

- View the full paper in a side-by-side study workspace.
- Generate 8 study goals.
- Ask questions and receive answers with page citations.

6. System Architecture

Frontend:

- Next.js with TypeScript.
- Tailwind CSS.
- Dark study interface.
- Components for search, paper cards, modal, PDF viewer, study goals, and chat.

Backend:

- FastAPI.
- arXiv Atom API for paper search.
- PyMuPDF for PDF text extraction and page rendering.
- Ollama for local LLM calls.
- Local file storage under backend/data/papers.

Local model:

- Ollama running locally.
- Current model used on this machine: qwen3.5:9b.
- Default environment variable support: OLLAMA_MODEL.

7. Data Pipeline

The pipeline works as follows:

- The user searches arXiv.
- The backend returns cleaned paper metadata.
- The user clicks Study with AI.
- The backend creates a safe local paper folder.
- The PDF is downloaded as paper.pdf.
- PyMuPDF extracts page-wise text.
- The system saves metadata.json, pages.json, and chunks.json.
- The PDF pages are rendered as PNG images for reliable viewing in the browser.

The chunking method is simple. Each page is split into chunks of about 1000 to 1800 words while preserving page numbers.

8. Model and Retrieval

The current assistant uses keyword overlap scoring for retrieval. When a user asks a question, the backend loads `chunks.json`, scores chunks against the question, and selects the top 4 chunks.

The selected chunks are placed into a grounded prompt for Ollama. The model is instructed to answer only from the paper context and cite page numbers such as [p. 2].

This is not a full RAG system with embeddings yet. That is acceptable for Milestone 1 because the retrieval method is transparent and easy to debug.

9. User Interface

The UI has two main views:

Home page:

- Search bar.
- Paper cards.
- Paper details modal.
- Recently viewed papers.
- Local bookmarks.

Study page:

- Full paper viewer on the left.
- AI study panel on the right.
- Study goals tab.
- Quick start tab.
- Chat box at the bottom.

The study page now shows the complete paper as scrollable rendered pages, not only the first page.

10. Preliminary Results

Test paper	Pages	Chunks	Study goals	Local answer
Attention Is All You Need	15	15	8 goals shown	Answer returned with page citations
RAG survey	TBD	TBD	Pending	Pending

The first test confirms that the full search to study flow works: search arXiv, prepare paper, render PDF pages, load study goals, ask a question, and receive a grounded answer.

11. Known Limitations

- Retrieval is keyword based, not embedding based.
- PDF extraction depends on the quality of the PDF text layer.
- Ollama can be slow on CPU or when the model is large.
- Study goals currently use fallback goals if the model takes too long.
- Local storage is single-user and file based.
- There is no deployment or authentication in this milestone.

12. Next Steps

- Add embedding based retrieval.
- Improve citation matching.
- Add better model timeout and streaming behavior.
- Add evaluation questions for study goals.
- Add a small results log for more test papers.
- Improve PDF page navigation and search.

13. Conclusion

Milestone 1 successfully builds a working local paper study assistant. The system is small, understandable, and useful enough to test with real arXiv papers. It proves the main workflow before adding more advanced research features.